```
AS SELECT * FROM change_log
WHERE when BETWEEN TO_DATE('01-JAN-2004','DD-Mon-YYYY')
 AND TO_DATE('31-JAN-2004','DD-Mon-YYYY');

SQL> describe january2004_log
 Name                      Null?    Type
 ------------------------- -------- ----------------
 LOG_ID                             NUMBER
 WHO                                VARCHAR2(64)
 WHEN                               TIMESTAMP(6)
 WHAT                               VARCHAR2(200)
```

The option NOLOGGING tells the database not to log the contents of the table to the redo log and not to log subsequent direct-path insert operations to the redo log. The COMPRESS option tells the database to add the data to the table using data compression, thus requiring less disk space. The TABLESPACE option tells the database where to store the table.

Creating a Temporary Table

You can create a temporary table whose contents are transitory and only visible to the session that inserted data into it. The definition of the table persists, but the data in a temporary table lasts only for either the duration of the transaction (ON COMMIT DELETE ROWS) or for the duration of the session (ON COMMIT PRESERVE ROWS). Here is an example:

```
CREATE GLOBAL TEMPORARY TABLE my_session
(category      VARCHAR2(16)
,running_count NUMBER
) ON COMMIT DELETE ROWS;
```

Programs can manipulate data in temporary tables or join them to permanent tables in the same manner as any other table.

Setting Default Values

You can set default values for the columns in your table. When subsequent INSERT statements do not explicitly populate these columns, the database assigns the default value to the column.

Having a default value does not ensure that the column will always have a value. An INSERT or an UPDATE statement can always explicitly set a column to NULL unless there is a NOT NULL constraint on the column.

See the section "Creating Constraints" later in this chapter for more information on creating or using constraints.

Default values can be any SQL expression that does not reference a PL/SQL function, other columns, or the pseudocolumns ROWNUM, NEXTVAL, CURRVAL, LEVEL, or PRIOR. To implement more complex rules in PL/SQL for assigning default values, you can use a BEFORE INSERT trigger.

See Chapter 7, "Managing Data with SQL, PL/SQL, and Utilities," for more information on creating triggers.

Default values are defined as part of a column specification. This definition can be made at table creation time, like this:

```
CREATE TABLE change_log
(log_id      NUMBER
,who         VARCHAR2(64) DEFAULT USER
,when        TIMESTAMP    DEFAULT SYSTIMESTAMP
,what        VARCHAR2(200)
) TABLESPACE users;
```

To define a default value after a table has been created, use the ALTER TABLE statement to modify the column specification, like this:

```
ALTER TABLE change_log MODIFY
who         VARCHAR2(64) DEFAULT USER;
```

Adding Comments to a Table or a Column

You can add descriptive comments to your tables and columns in order to better describe the content or usage of these database objects. Comments can be a maximum of 4,000 bytes in length and can have embedded white space and punctuation.

Use the COMMENT ON statement to assign a comment to either a table or a column, like this:

```
COMMENT ON TABLE change_log IS
  'This table is where you record changes
 to the configuration of the DEMO system';

COMMENT ON COLUMN change_log.log_id IS
'System generated key for change log table
 Populated with the change_seq sequence.';
```

The comment must be enclosed in single quotes, but can span physical lines. To display the comments on a table, query the DBA_TAB_COMMENTS, ALL_TAB_COMMENTS, or USER_TAB_COMMENTS data dictionary views:

```
SELECT owner, table_name, comments
FROM all_tab_comments
```

```
WHERE table_name = 'CHANGE_LOG';


OWNER  TABLE_NAME  COMMENTS
------ ----------- ---------------------------------------
CHIP   CHANGE_LOG  This table is where you record changes
                   to the configuration of the DEMO system
```

To display the comments on a column, query the DBA_COL_COMMENTS, ALL_COL_COMMENTS, or USER_COL_COMMENTS data dictionary views:

```
SELECT table_name, column_name, comments
FROM user_col_comments
WHERE table_name = 'CHANGE_LOG'
AND   column_name = 'LOG_ID';


TABLE_NAME COLUMN COMMENTS
---------- ------ ----------------------------------------
CHANGE_LOG LOG_ID System generated key for change log table
                  Populated with the change_seq sequence.
```

Renaming a Table

You can use the RENAME statement to change the name of a table, view, sequence, or private synonym—objects that share a namespace with tables. The syntax is RENAME *old_name* TO *new_name*. When you rename a table, the database invalidates all objects that depend on (refer to) the table, such as views, procedures, or synonyms. The database automatically alters associated indexes, grants, and constraints to reference the new name.

To change the name of the change_log table to demo_change_log, execute this:

```
RENAME change_log TO demo_change_log;
```

Another way to rename a table is with the RENAME TO clause of the ALTER TABLE statement. For example:

```
ALTER TABLE change_log RENAME TO demo_change_log;
```

Neither syntax is preferred; both semantics are fully supported.

Adding and Dropping Columns in a Table

One task that you will need to perform while managing tables is adding new columns to an existing table. Use the ALTER TABLE statement to add columns. When adding a single column, use the syntax ALTER TABLE *table_name* ADD *column_spec*.

When you add multiple columns to a table, enclose a comma-delimited list of column specifications with parentheses. The column specification includes the column name, the column's datatype, and any default value that the column will have.

For example, to add a column named HOW to the change_log table, execute the following SQL:

```
ALTER TABLE change_log ADD how VARCHAR2(45);
```

To add the two columns—HOW and WHY—to the change_log table, use the syntax with parentheses, like this:

```
ALTER TABLE change_log ADD
(how     VARCHAR2(45)
,why     VARCHAR2(60)
);
```

To remove a column from a table, use the ALTER TABLE DROP COLUMN statement, as in this example:

```
ALTER TABLE change_log DROP COLUMN how;
```

To drop multiple columns, you don't use the keyword COLUMN, and instead enclose the comma-delimited list of columns in parentheses:

```
ALTER TABLE change_log DROP (how,why);
```

Modifying Columns

You may need to occasionally make changes to the columns of a table—increase or decrease the size of a column, rename a column, or assign a default value to a column. You use the ALTER TABLE MODIFY statement to make these column-level changes. As with the ADD and DROP options, you have two syntactical options: one for modifying a single column and one for modifying multiple columns.

To make changes to a single column, you specify the column name together with the new characteristics. For example, to change the column WHAT from VARCHAR2(200) to VARCHAR2(250), you execute:

```
ALTER TABLE change_log MODIFY what VARCHAR2(250);
```

To change multiple columns, enclose a comma-delimited list of modified column specs in parentheses, like this:

```
ALTER TABLE change_log MODIFY
(what    VARCHAR2(250)
,who     VARCHAR2(50)  DEFAULT user
);
```

Viewing the Attributes of a Table

You can use several data dictionary views to display the attributes of a table. The first place to start is usually with the DESCRIBE command from SQL*Plus or *i*SQL*Plus:

```
SQL> describe employees
 Name                      Null?    Type
 ------------------------- -------- ----------------
 EMPLOYEE_ID               NOT NULL NUMBER
 HIRE_DATE                 NOT NULL DATE
 FIRST_NAME                         VARCHAR2(42)
 LAST_NAME                          VARCHAR2(42)
 PAYROLL_ID                         VARCHAR2(10)
 DEPT_NBR                           NUMBER
 MANAGER                            NUMBER
```

The DESCRIBE command displays the column names, datatypes, and nullity of each column. To see the table physical attributes, query the ALL_TABLES or USER_TABLES dictionary view, like this:

```
SELECT * FROM user_tables
WHERE table_name = 'EMPLOYEES';
```

To see what constraints are declared on a table, use the ALL_CONSTRAINTS and ALL_CONS_COLUMNS views, like this:

```
SELECT constraint_name,constraint_type ,r_constraint_name
FROM user_constraints
WHERE table_name = 'EMPLOYEES';

                               CONS
CONSTRAINT_NAME                TYPE   R_CONSTRAINT_NAME
------------------------------ ------ ---------------------
NN_EMP_ID                      C
SYS_C005286                    C
EMPLOYEES_PK                   P
UNIQ_PAYROLL_ID                U
EMP_DEPT_FK                    R      DEPARTMENT_PK
MGR_EMP_FK                     R      EMPLOYEES_PK
HIRE_DATE_CHECK                C
```

The CONSTRAINT_TYPE column indicates the kind of constraint:

- C is for Check
- P is for Primary Key

- R is for Referential (or Foreign Key)
- U is for Unique

NOT NULL constraints are stored both as a column attribute and as a check constraint.

The SEARCH_CONDITION column of the USER_CONSTRAINTS view is only applicable for CHECK constraints:

```
SELECT search_condition
FROM user_constraints
WHERE constraint_name = 'SYS_C005286';

SEARCH_CONDITION
-----------------------------------------------
"HIRE_DATE" IS NOT NULL
```

The columns participating in a FOREIGN KEY constraint can be found in the ALL_CONS_COLUMNS view, like this:

```
SELECT column_name, position
FROM all_cons_columns
WHERE constraint_name = 'EMP_DEPT_FK';

COLUMN_NAME                     POSITION
------------------------------ ----------
DEPT_NBR                                1
```

Viewing the Contents of a Table

To view the contents of a table, simply select the columns of interest from the table. You can use the column list reported in the DESCRIBE command to limit your column selection or you can specify * for the column list to view all columns in the table. Use a WHERE clause to filter the rows, like this:

```
SELECT * FROM employees;

SELECT employee_id, first_name, last_name, hire_date
FROM employees
WHERE hire_date > SYSDATE - 90
;
```

Working With Constraints

Constraints enforce business rules in the database. In other words, they limit the acceptable data values for a table. Constraints are optional schema objects that depend on tables. Although you can have a table without any constraints, you cannot create a constraint without a table.

Oracle lets you create several types of constraints on your tables to enforce your business rules, including the following:

- NOT NULL
- UNIQUE
- PRIMARY KEY
- REFERENTIAL
- CHECK

You can create constraints together with the table in the CREATE TABLE statement. After you create a table, you add or remove a constraint from a table with an ALTER TABLE statement. You specify the constraint information with either the in-line syntax as a column attribute or the out-of-line syntax as part of the table definition. Constraints do not require a name; if you do not name the constraint, Oracle generates one for you. However, the generated names are simply numbers prefixed with SYS_C and may not be very meaningful.

The following sections describe each of the constraint types in detail.

Working with *NOT NULL* Constraints

By default, all columns in a table allow NULL as a valid value. A NULL represents unknown or nonexistent information. Some business rules can be enforced with a NOT NULL constraint. For example, an employee may not be considered a valid employee if their hire date is not known. You enforce this business rule by placing a NOT NULL constraint on the hire_date column of the employees table. Any INSERT or UPDATE statements fail if the protected column does not have a value.

NOT NULL constraints must be declared together with the column definition using in-line syntax.

Here is an example using the NOT NULL constraint:

```
CREATE TABLE employees
(employee_id    NUMBER  CONSTRAINT nn_emp_id  NOT NULL
,hire_date      DATE                          NOT NULL
,first_name     VARCHAR2(42)
,last_name      VARCHAR2(42)
);
```

Working with *UNIQUE* Constraints

A UNIQUE constraint ensures that each occurrence of the columns protected by this constraint is different from all other occurrences in the table. UNIQUE constraints cannot be created on columns of type CLOB, NCLOB, BLOB, LONG, LONG RAW, or TIMESTAMP WITH TIMEZONE.

Here is how you create an employees table that has a UNIQUE constraint on the payroll_id column using the out-of-line syntax:

```
CREATE TABLE employees
(employee_id    NUMBER       NOT NULL
```

```
,hire_date      DATE         NOT NULL
,first_name     VARCHAR2(42)
,last_name      VARCHAR2(42)
,payroll_id     VARCHAR2(10)
,CONSTRAINT uniq_payroll_id UNIQUE (payroll_id)
);
```

Using the in-line syntax, the statement looks like this:

```
CREATE TABLE employees
(employee_id    NUMBER     NOT NULL
   CONSTRAINT uniq_payroll_id UNIQUE
,hire_date      DATE       NOT NULL
,first_name     VARCHAR2(42)
,last_name      VARCHAR2(42)
,payroll_id     VARCHAR2(10)
);
```

No two rows in this table can have the same value in `payroll_id`. `NULL` values do not count as a distinct value, so this `employees` table can have multiple rows with a `NULL payroll_id`. To ensure that `payroll_id` is always present, you need a `NOT NULL` constraint.

The database uses an index to help enforce this constraint. The index is usually a unique index, and if you create the `UNIQUE` constraint together with the table, the database automatically creates a unique index on the columns protected by the `UNIQUE` constraint, and the name of the index defaults to the name of the constraint. To assign attributes to this index, take advantage of the `USING INDEX` clause, like this:

```
CREATE TABLE employees
(employee_id    NUMBER       NOT NULL
,hire_date      DATE         NOT NULL
,first_name     VARCHAR2(42)
,last_name      VARCHAR2(42)
,payroll_id     VARCHAR2(10)
,CONSTRAINT uniq_payroll_id UNIQUE (payroll_id)
  USING INDEX TABLESPACE indx
);
```

You can add a `UNIQUE` constraint after the table is built by using an `ALTER TABLE` statement, like this:

```
ALTER TABLE employees ADD
CONSTRAINT uniq_payroll_id UNIQUE (payroll_id)
  USING INDEX TABLESPACE indx
;
```

When adding a UNIQUE constraint to an existing table, Oracle raises an exception, and the statement fails if there are any duplicate keys (that is, violations of the constraint). To assist you in identifying any duplicates, you can specify an EXCEPTIONS INTO clause. First, create an exceptions table with the utlexcpt.sql (using ROWID) or utlexpt1.sql (using UROWID) script located in the *oracle_home*/rdbms/admin directory. These scripts create a table named EXCEPTIONS that contains columns for the ROWID, owner, table_name, and constraint that is in violation. You can use this information to fix any data that violates the constraint and then try the ALTER statement again to create the constraint.

Here is an example:

```
ALTER TABLE employees ADD
   CONSTRAINT uniq_payroll_id UNIQUE (payroll_id)
   USING INDEX TABLESPACE indx
   EXCEPTIONS INTO exceptions;
```

Working with *PRIMARY KEY* Constraints

Primary keys are defined in a relational model as being the unique identifier for any tupple in an entity. PRIMARY KEY constraints enforce the validity of a primary key, so a table can have only one PRIMARY KEY constraint. A PRIMARY KEY constraint implicitly includes both NOT NULL constraints on each column in the key as well as a UNIQUE constraint, enforced with an index, on all columns in the key. You can create a PRIMARY KEY constraint at the same time that you create a table.

Here is an example of creating a PRIMARY KEY constraint together with a table, using out-of-line syntax:

```
CREATE TABLE employees
(employee_id    NUMBER       NOT NULL
,hire_date      DATE         NOT NULL
,first_name     VARCHAR2(42)
,last_name      VARCHAR2(42)
,payroll_id     VARCHAR2(10)
,CONSTRAINT employees_pk PRIMARY KEY (employee_id)
  USING INDEX TABLESPACE indx
);
```

As with UNIQUE constraints, PRIMARY KEY constraints cannot be created on columns of type CLOB, NCLOB, BLOB, LONG, LONG RAW, or TIMESTAMP WITH TIMEZONE.

Working with *FOREIGN KEY* Constraints

FOREIGN KEY constraints are also known as referential integrity constraints because they enforce referential integrity. FOREIGN KEY constraints enforce referential integrity by ensuring that data values referenced in one table are defined in another table. FOREIGN KEY constraints tie these two tables together in a parent/child or referenced/dependent relationship.

When you declare a FOREIGN KEY constraint, you identify the columns in one table whose values must also appear in the primary key or unique key of another table. The table with the primary or unique key is known as the parent or referenced table, and the table with the FOREIGN KEY constraint is known as the child or dependent table.

As with UNIQUE and PRIMARY KEY constraints, FOREIGN KEY constraints cannot be created on columns of type CLOB, NCLOB, BLOB, LONG, LONG RAW, or TIMESTAMP WITH TIMEZONE.

Here is an example of creating a parent table (DEPARTMENTS) and child table (EMPLOYEES) with a PRIMARY KEY constraint on the parent and a FOREIGN KEY constraint on the child table, using out-of-line syntax:

```
CREATE TABLE departments
(dept_nbr      NUMBER   NOT NULL
   CONSTRAINT department_pk PRIMARY KEY
,dept_name     VARCHAR2(32)
,manager_id    NUMBER
);

CREATE TABLE employees
(employee_id    NUMBER       NOT NULL
,hire_date      DATE         NOT NULL
,first_name     VARCHAR2(42)
,last_name      VARCHAR2(42)
,payroll_id     VARCHAR2(10)
,dept_nbr       NUMBER
,CONSTRAINT uniq_payroll_id UNIQUE (payroll_id)
  USING INDEX TABLESPACE indx
,CONSTRAINT employees_pk PRIMARY KEY (employee_id)
  USING INDEX TABLESPACE indx
,CONSTRAINT emp_dept_fk FOREIGN KEY (dept_nbr)
  REFERENCES departments(dept_nbr)
);
```

In this example, each employee belongs to a department, so we put a DEPT_NBR column in the EMPLOYEES table, which will hold each employee's department number. The DEPARTMENTS table defines all the valid department numbers to ensure that DEPT_NBR values appearing in the EMPLOYEES table are defined in the DEPARTMENTS table—in essence that an employee belongs to a valid department. You implement this relationship or rule with a FOREIGN KEY constraint. By default, FOREIGN KEYs allow NULLs.

By default, the database raises an exception and does not allow you to delete rows from a parent table if those rows are referenced in the child table. If this behavior isn't what you want, you can tell the database to automatically maintain referential integrity in a couple of ways: by

deleting the child rows and specifying ON DELETE CASCADE or by setting the columns in the child table to NULL with the ON DELETE SET NULL clause, like this:

```
ALTER TABLE employees
   ADD CONSTRAINT emp_dept_fk FOREIGN KEY (dept_nbr)
     REFERENCES departments(dept_nbr) ON DELETE CASCADE;

ALTER TABLE departments ADD CONSTRAINT
   dept_mgr_fk FOREIGN KEY (manager_id) REFERENCES
     employees(employee_id) ON DELETE SET NULL;
```

The first statement tells the database that deleting a department should cause a cascading deletion of that department's employees. The second statement tells the database that deleting an employee who is a department manager should cause that department's MANAGER_ID column to revert to NULL.

A Self-Referencing Foreign Key

The parent and child tables do not always have to be separate tables; they can be separate columns of the same table. This configuration is known as a self-referencing foreign key. An example of a self-referencing foreign key can be added to the EMPLOYEES table used in the previous section. The business rule that will be enforced requires that each employee report to a manager and also that the manager be a valid employee. To add this rule to the EMPLOYEES table, add the MANAGER column together with a FOREIGN KEY constraint on which it references the EMPLOYEES table, like this:

```
ALTER TABLE employees ADD
   (manager    NUMBER
   ,CONSTRAINT mgr_emp_fk FOREIGN KEY (manager)
      REFERENCES employees(employee_id)
      ON DELETE SET NULL
   );
```

Deferred Constraint Checking

It's possible that your programs might temporarily violate a FOREIGN KEY constraint without really violating the underlying business rule if a program adds data to both tables participating in a FOREIGN KEY constraint within the scope of a single transaction.

For example, if you hire a new employee and create a new department for that person to manage, you need to add a row to both the EMPLOYEES table (which references the new department) as well as the DEPARTMENTS table (which references the new employee). Although this temporary violation will not go against the intent of the business rule, you will need to create the constraints with some additional options, like this:

```
ALTER TABLE employees
   ADD CONSTRAINT emp_dept_fk FOREIGN KEY (dept_nbr)
```

```
    REFERENCES departments(dept_nbr) ON DELETE CASCADE
    DEFERRABLE;

ALTER TABLE departments ADD CONSTRAINT
   dept_mgr_fk FOREIGN KEY (manager_id) REFERENCES
    employees(employee_id) ON DELETE SET NULL
    DEFERRABLE INITIALLY DEFERRED;
```

By default, the database checks that a FOREIGN KEY constraint is satisfied at the end of each statement. You define this behavior with the keywords INITIALLY IMMEDIATE. Also by default, the database will not allow programs to defer constraint checking to the end of the transaction. You define this behavior with the keywords NOT DEFERRABLE.

When you create a constraint, you can tell the database to allow either immediate or deferred constraint checking by specifying the keyword DEFERRABLE. If you normally want a DEFERRABLE constraint to be deferred, create it with the INITIALLY DEFERRED option. Only DEFERRABLE constraints can be set to INITIALLY DEFERRED. Once you create a constraint, you cannot change its deferability (for example, from NOT DEFERRABLE to DEFERABLE); instead, you must drop and re-create the constraint with the new specification.

Working with *CHECK* Constraints

CHECK constraints verify that a column or set of column values meet a specific condition that must evaluate to a Boolean. If the condition evaluates to FALSE, the database raises an exception, and the INSERT or UPDATE statement fails. The condition cannot include subqueries, references to other tables, or calls to functions that are not deterministic. A function is deterministic if it always returns the same result when passed the same input parameters. Examples of deterministic functions include SQRT, TO_DATE, and SUBSTR. The functions SYSDATE, USER, and DBTIMEZONE are not deterministic. The condition must be a Boolean SQL expression enclosed in parentheses.

Add a CHECK constraint to ensure that every employee's hire date is later than the company's founding date, like this:

```
ALTER TABLE employees ADD CONSTRAINT
   validate_hire_date CHECK
   (hire_date > TO_DATE('15-Apr-1999','DD-Mon-YYYY'));
```

Modifying Constraints

Once created, constraints can be dropped, disabled (temporarily not enforced), enabled (enforced again), or renamed. You make these changes to constraints using an ALTER TABLE statement. Take care in disabling UNIQUE or PRIMARY KEY constraints because disabling these constraints results in the supporting index being dropped (unless you also specify KEEP INDEX.

To drop a constraint, use an ALTER TABLE statement with the constraint name, like this:

```
ALTER TABLE employees DROP CONSTRAINT validate_hire_date;
```

Because there can be only one PRIMARY KEY constraint on a table, you can drop it by simply specifying DROP PRIMARY KEY without actually using the constraint's name. If FOREIGN KEY constraints reference your PRIMARY KEY or UNIQUE constraint, you need to drop these dependent constraints before or in conjunction (using the CASCADE keyword) with the PRIMARY KEY constraint:

```
ALTER TABLE employees DROP PRIMARY KEY CASCADE;
```

To rename a constraint, give the old and new names:

```
ALTER TABLE employees
   RENAME CONSTRAINT validate_hire_date TO hire_date_check;
```

When bulk loading data into a table, it is often more efficient to disable FOREIGN KEY and CHECK constraints, load the data, and then re-enable these constraints, like this:

```
ALTER TABLE employees DISABLE CONSTRAINT mgr_emp_fk;
-- bulk load the table
ALTER TABLE employees ENABLE  CONSTRAINT mgr_emp_fk;
```

Disabling either a PRIMARY KEY or a UNIQUE constraint may drop the supporting index and therefore may not be desirable for a load process that uses that index.

Working with Indexes

Indexes are optional data structures built on tables. Indexes can improve data retrieval performance by providing a direct access method instead of the default full table scan retrieval method. You can build Btree or bitmap indexes on one or more columns in a table. An *index key* is defined as one data value stored in the index. A Btree index sorts the keys into a binary tree and stores these keys together with the table's ROWIDs. In a bitmap index, a bitmap is created for each key. There is a bit in each bitmap for every ROWID in the table, forming the equivalent of a two-dimensional matrix. The bits are set if the corresponding row in the bitmap exists.

Btree indexes are the default index type, can be unique or non-unique, and are appropriate for medium- to high-*cardinality* columns—those having many distinct values. Btree indexes support row-level locking and so are appropriate for multiuser, transactional applications. The indexes supporting a PRIMARY KEY or UNIQUE constraints are Btree indexes.

Bitmap indexes, on the other hand, are best for multiple combinations of low- to medium-cardinality columns (you cannot create a unique bitmap index), and they do not support row-level locking. Bitmap indexes are best in environments in which changes to data are limited and controlled, such as many data warehousing applications. Because bitmap indexes cannot efficiently make changes to the indexed data, they are often dropped prior to data loading and then re-created after a data load.

In the following section, we will show you how to create, drop, and manage indexes.

Creating and Dropping Indexes

To create an index, you first need a table. In the CREATE INDEX statement, you tell the database the name of the new index, which table to create the index on, and which columns to include. If multiple columns will be included, use a comma-delimited list.

To create a Btree index on the DEPT_NBR column of the EMPLOYEES table used in the preceding sections, use a CREATE INDEX statement, like this:

```
CREATE INDEX emp_dept_nbr ON employees (dept_nbr)
   TABLESPACE indx;
```

A unique index requires the additional keyword UNIQUE, like this:

```
CREATE UNIQUE INDEX dname_uix ON departments (dept_name);
```

If you frequently access employees by seniority, you can create a multicolumn index on both department number and hire date, like this:

```
CREATE INDEX emp_seniority ON
   employees (dept_nbr, hire_date)
   TABLESPACE indx;
```

To create three single-column bitmap indexes on the STATE, REGION, and METRO_AREA columns of the GEOGRAPHY table, execute the following:

```
CREATE BITMAP INDEX state_bix  ON geography (state);
CREATE BITMAP INDEX region_bix ON geography (region);
CREATE BITMAP INDEX metro_bix  ON geography (metro_area);
```

To drop an index, use a DROP INDEX statement, like these:

```
DROP INDEX emp_seniority;
DROP INDEX state_bix;
```

Managing Indexes

You can perform several maintenance actions on an index, including rebuilding an index, moving it to a new tablespace, coalescing it, or renaming the index. All these actions are performed with different clauses of an ALTER INDEX statement.

To rebuild an index, which will shrink its size and possibly reduce the Btree depth (making it more efficient), use a REBUILD clause, like this.

```
ALTER INDEX emp_seniority REBUILD;
```

To move an index from one tablespace to another, you specify a new tablespace in conjunction with REBUILD:

```
ALTER INDEX uniq_payroll_id REBUILD TABLESPACE hr_indx;
```

Coalescing an index is like a quick-and-dirty rebuild. Instead of re-creating the Btree, a coalesce combines entries in nearby leaf blocks with the intent of freeing space from some of the leaf blocks. The space and resources required for a coalesce is much less than the full rebuild. But when you coalesce an index, you cannot move the index or reduce the depth of the Btree. Here is an example of coalescing an index:

```
ALTER INDEX uniq_payroll_id COALESCE;
```

Renaming an index is beneficial if you have a poorly named index. It also comes in handy if you need to take care of some high-availability maintenance actions that include staging the new version of an index using a temporary name and then in a short deployment window renaming the old and new indexes. Here is an example of renaming an index:

```
ALTER INDEX sys_c001428 RENAME TO employee_pk;
```

Working with Views

Views are virtual tables consisting of a stored query. A view is created with a query that incorporates one or more base tables. Although views are often used for selecting data, some views can also be updated, deleted, or inserted into. You can use views to restrict access to a subset of a table, or you can make an awkward join appear as a simple table, reducing a query's complexity.

The data dictionary is a good example of a collection of complex joins appearing as more simple tables. The `ALL_CONSTRAINTS` view, for example, is a complex nine-table join that also includes two subqueries. Trying to navigate the base tables used in the dictionary views is a bit daunting, but the dictionary views are much more usable because they hide a great deal of the underlying complexity.

Likewise, some data is deemed too sensitive for most access, such as the passwords associated with a database link. The data is stored in the underlying base table `link$`, but not revealed in the dictionary views `DBA_DB_LINKS` or `ALL_DB_LINKS`. Because access to the base table is restrictive, but the dictionary views are widely accessed, the database can hide the sensitive data in the base tables.

To create a view named `empv` that is based on a combination of data in both the `EMPLOYEES` and `DEPARTMENTS` tables, you can execute the following:

```
CREATE OR REPLACE VIEW empv
(employee_name
,department
,manager
,hire_date
) AS SELECT
 E.first_name||' '||E.last_name
,D.dept_name
,M.first_name||' '||M.last_name
,E.hire_date
```

```
FROM employees   E
    ,departments D
    ,employees   M
WHERE E.dept_nbr = D.dept_nbr
 AND  D.manager_id = M.employee_id
;
```

The OR REPLACE keywords tell the database to replace the view definition if it already exists. If the OR REPLACE keyword is not included, the statement fails if the view already exists. The column list in the parentheses is the list of column names as they appear in the view and correspond positionally with the column expressions in the query—the first expression the query maps to the first column name for the view, the second expression in the query, to the second column name, and so on. You can choose not to include salary or commission in the view definition and only grant privileges on the view, thus restricting access to sensitive data.

To remove a view from the database, use a DROP VIEW statement, like this:

```
DROP VIEW empv;
```

Working with Sequences

Sequences are schema objects that generate unique integers. They are frequently used to generate primary key values.

The CREATE SEQUENCE statement creates new sequences. You must, at a minimum, give the sequence a name. You can also assign the following attributes to the sequence in the CREATE statement:

START WITH Defines the initial value for the sequence.

INCREMENT BY Defines how the subsequent sequence numbers will be generated. The default is 1, and valid values are nonzero integers of less than 29 digits. A negative INCREMENT BY results in a descending sequence that generates progressively lower numbers. A positive INCREMENT BY results in an ascending sequence that generates progressively higher numbers.

MAXVALUE Defines the highest value that the sequence can generate. The default is the special keyword NOMAXVALUE, which evaluates to 10^{27} for an ascending sequence and to –1 for a descending sequence.

MINVALUE Defines the lowest value that the sequence can generate. The default for MINVALUE is the special keyword NOMINVALUE, which evaluates to -10^{26} for an ascending sequence and to –1 for a descending sequence.

CACHE Defines how many values will be preallocated and held in memory. The default is 20, and the valid values range from 2 to the number returned from the formula:

```
(CEIL (MAXVALUE - MINVALUE)) / ABS(INCREMENT BY)
```

To create a sequence called `employees_seq` that generates unique integers, starting with 100500 and incrementing by 1, use the following statement:

```
CREATE SEQUENCE employees_seq START WITH 100500 NOMAXVALUE NOMINVALUE;
```

To request the next integer from the sequence, reference the pseudocolumn `NEXTVAL` in a SQL statement, like this:

```
SELECT employees_seq.nextval FROM dual;
```

These values can be used in `INSERT` statements or in `UPDATE` statements, like this:

```
INSERT INTO employees (employee_id, hire_date)
   SELECT employees_seq.NEXTVAL, hired_date
   FROM merged_employees;
```

You can use the `ALTER SEQUENCE` statement to modify the `INCREMENT BY`, `MAXVALUE`, `MINVALUE`, `CYCLE`, and `CACHE` attributes, but not the `START WITH` attribute, like this:

```
ALTER SEQUENCE employees_seq INCREMENT BY -1;
```

If you need to reset the next value that a sequence will generate, drop it, re-create it, and then re-grant any privileges on it. To drop a sequence, use the `DROP SEQUENCE` statement, like this:

```
DROP SEQUENCE employee_seq;
```

Summary

In this chapter, you learned about the purpose of tablespaces and datafiles. You found out how to create and manage tablespaces as well as how Oracle stores some schema objects as segments in extents and data blocks.

You learned how to create and modify tables, indexes, constraints, views, and sequences. We covered deferred constraint checking and how to configure foreign key constraints to support either deferrable or not deferrable implementations.

Exam Essentials

Know the relationship of datafiles to tablespaces. Tablespaces are built on one or more datafiles—bigfile tablespaces on a single datafile and smallfile tablespaces on one or more datafiles.

Understand the statements needed to create, modify, and drop tablespaces. Use a `CREATE TABLESPACE`, `ALTER TABLESPACE`, or `DROP TABLESPACE` statement to create, modify, and drop a tablespace.

Know how to take tablespaces offline and what consequences the offline immediate option poses. Use an `ALTER TABLESPACE` statement to take a tablespace offline or bring it online. If you use the offline immediate option, you must perform media recovery when you bring it back online.

Know how to use the EM Database Control to view tablespace information. The EM Database Control can be used to view tablespace information as well as perform various administrative tasks. A working knowledge of this tool is required.

Know the difference between segment space management and extent management. Extent management deals with segment-level space allocations, and segment space management deals with data block–level space allocations.

Know what initialization parameter controls Oracle Managed Files (OMF) placement. The `DB_CREATE_FILE_DEST` parameter tells the database where to place Oracle Managed Files.

Know what constitutes a valid name for a database object. Database object names can be a maximum of 30 bytes, must begin with a letter, and can be composed of letters, numbers, and the special characters underscore, dollar sign, or pound sign. If the database name is enclosed in double quotes, it can be mixed case and contain other punctuation marks.

Know what a namespace is and how it determines which objects can have the same name. Database object names must be unique within a namespace. One namespace is shared by tables, views, sequences, private synonyms, procedures, functions, packages, materialized views, and user-defined types. There are separate namespaces for each of the following: indexes, constraints, clusters, database triggers, private database links, and dimensions.

Know the different types of constraints and which have dependencies with others. There are `CHECK`, `NOT NULL`, `UNIQUE`, `PRIMARY KEY`, and `FOREIGN KEY` constraints. A `PRIMARY KEY` constraint implicitly includes `NOT NULL` and `UNIQUE` constraints. A `FOREIGN KEY` constraint must refer to a `PRIMARY KEY` or `UNIQUE` constraint.

Know the types of indexes and when they are appropriate. There are Btree and bitmap indexes. Btree indexes are medium- to high-cardinality columns in applications in which data can change frequently. Bitmap indexes are best for low- to medium-cardinality columns in applications that control data changes, usually in batches.

Review Questions

1. Which of the following statements about tablespaces is true?
   - **A.** A tablespace is the physical implementation of logical structure called a namespace.
   - **B.** A tablespace can hold the objects of only one schema.
   - **C.** A bigfile tablespace can have only one datafile.
   - **D.** The `SYSAUX` tablespace is an optional tablespace only created if you install certain database options.
2. Automatic segment space management on the tablespace causes which of the following table attributes in that tablespace to be ignored?
   - **A.** The whole storage clause
   - **B.** `NEXT` and `PCTINCREASE`
   - **C.** `BUFFERPOOL` and `FREEPOOL`
   - **D.** `PCTFREE` and `PCTUSED`
3. Which objects share the same namespace and therefore cannot have the same name?
   - **A.** Tables and indexes
   - **B.** Tables and procedures
   - **C.** Tables and constraints
   - **D.** Tables and triggers
4. Which is not a type of segment that is stored in a tablespace?
   - **A.** Undo
   - **B.** Redo
   - **C.** Permanent
   - **D.** Temporary
5. With which parameters do you specify unlimited datafile growth?
   - **A.** `MAXSIZE UNLIMITED`
   - **B.** `UNLIMITED GROWTH`
   - **C.** `MAXEXTENTS UNLIMITED`
   - **D.** Datafile size cannot change.
6. Which of the following is not a character datatype that can be used in a table column definition?
   - **A.** `char`
   - **B.** `varchar`
   - **C.** `nvarchar2`
   - **D.** `string`

7. With which numeric datatype can you represent infinity?
   - **A.** `double`
   - **B.** `float`
   - **C.** `binary_float`
   - **D.** Infinity cannot be represented in the database.

8. A table name can never include the special meta-character dollar sign ($).
   - **A.** True
   - **B.** False
   - **C.** Only if the table name is enclosed in double quotes
   - **D.** Only if the table name is enclosed in single quotes

9. Which of the following column specifications results in a column that will store time values down to the microsecond decimals of precision?
   - **A.** `timestamp(3)`
   - **B.** `time(3)`
   - **C.** `datetime(3)`
   - **D.** `date(3)`

10. Which type of index can specify as a unique index?
    - **A.** Bitmap
    - **B.** Heap organized
    - **C.** Btree
    - **D.** XOR

11. Which operation can you not do to a table that is created with the following SQL statement?

```
CREATE TABLE properties
("Location"  NUMBER primary key
,value       NUMBER(15)
,lot         varchar2(12)
,constraint positive_value check
   (value > 0)
);
```

    - **A.** Rename the primary key to properties.
    - **B.** Insert a null into the value column.
    - **C.** Add a column named owner.
    - **D.** Rename the index supporting primary key to properties.
    - **E.** None of the above.

12. Which of the statements is true regarding the table created with the following SQL statement?

```
CREATE TABLE autos
(vin         VARCHAR2(64) primary key
,style       VARCHAR2(15) default 'TUDOR'
,year        char(4)
,make        varchar2(12)
,model       varchar2(30)
);
```

A. `MAKE` is a reserved word; the `CREATE TABLE` will fail.

B. The column style will always have a value.

C. There is no index on this table.

D. The column style can have a `NULL` value.

13. Which constraint-checking model is the default?

A. Initially immediate and deferrable

B. Initially immediate and not deferrable

C. Initially deferred and not immediately

D. Initially deferrable and not immediate

14. Which statement on views is true?

A. A view can only be on one base table, although that base table can be joined to itself.

B. A view cannot be created with the same name and columns as the base table.

C. Inserts into a view are not allowed.

D. Privileges on a view can be different from those on the base table.

15. What can tablespaces be used for?

A. To organize tables and indexes into manageable groupings

B. To make sure that data stored in the tablespace does not change

C. To move data from one database to another

D. All of the above

16. Which allocation unit is the smallest?

A. Datafile

B. Extent

C. Data block

D. Segment

17. Which is a valid tablespace extent management specification?

A. Automatic

B. Local

C. Manual

D. Temporary

18. Which of the following is not a valid Oracle10*g* datatype?

A. `timestamp with local timezone`

B. `binary`

C. `blob`

D. `urowid`

19. How do you specify that a temporary table will be emptied at the end of a user's session?

A. Create the temporary table with the `ON COMMIT PRESERVE ROWS` option.

B. Create the temporary table with the `ON DISCONNECT PRESERVE ROWS` option.

C. Create the temporary table with the `ON DISCONNECT PURGE ROWS` option.

D. Create the temporary table with the `ON COMMIT DELETE ROWS` option.

20. How can you change the comment assigned to the columns in a table?

A. Use the `ALTER TABLE MODIFY COLUMN` statement.

B. Use the `COMMENT ON TABLE` statement.

C. Use the `RENAME` statement.

D. Use the `COMMENT ON COLUMN` statement.

Answers to Review Questions

1. C. Bigfile tablespaces are new to Oracle10*g* and can have only a single datafile. The traditional or smallfile tablespace can have many datafiles.

2. D. Segment space management refers to free space management, with automatic segment space management using bitmaps instead of `FREELISTS`, `PCTFREE`, and `PCTUSED`.

3. B. Indexes, constraints, and triggers all have separate namespaces. Tables share a namespace with views, sequences, private synonyms, procedures, functions, packages, materialized views, and user-defined types. Objects within the same schema sharing a namespace must have unique names.

4. B. Redo information is not stored in a segment; it is stored in the redo logs. Undo segments are stored in the undo tablespace, temporary segments are in the temporary tablespace, and permanent segments go into all the other tablespaces.

5. A. The autoextend `MAXSIZE` parameter tells Oracle how large a data or temp file can grow to. `UNLIMITED` specifies no bounds to the automatic growth.

6. D. The character datatypes include `char`, `nchar`, `varchar`, `varchar2`, `nvarchar2`, and `long` but do not include `string`.

7. C. With Oracle 6 through Oracle 9*i*, infinity could not be represented in the database. With Oracle10*g*, however, `binary_float` and `binary_double` can represent infinity, not a number (NAN), as well as several other special values.

8. B. Objects in an Oracle10*g* database can always include letters, numbers, and the characters $, _, and # (dollar sign, underscore, and pound sign). Names can include any other character only if they are enclosed in double quotes. The character dollar sign is not a special meta-character in an Oracle10*g* database.

9. A. There is no `time` or `datetime` datatype in Oracle10*g*. The `date` datatype cannot store subsecond granularity. The `timestamp` datatype stores subsecond granularity, defaulting to six digits of precision.

10. C. Heap organized is a table that is not index organized; it is not an index type. XOR is bitwise function and not an index type. Bitmap is an index type, but cannot be used for a unique index. Btree indexes make fine unique indexes.

11. E. You can rename both a constraint and an index to the same name as a table—they are in separate namespaces. Columns can be added, and owner is a valid column name. If the check constraint condition evaluates to `FALSE`, the data value will not be allowed; if the condition evaluates to either `TRUE` or `NULL`, the value is allowed.

12. D. A default clause ensures that the column does not contain a `NULL` after an insert, but not after an update. `MAKE` is not a reserved word, and an index will be created on the primary key.

13. B. Constraints can be created as deferrable and initially deferred, but deferred constraint checking is not the default.

14. D. Views can be created on one or more base table. Views share the same namespace as tables and therefore cannot have the same name; columns, however, can be named the same as the base table. `SELECT`, `INSERT`, `UPDATE`, and `DELETE` are all valid operations on a view. One of the uses for a view is to hide portions of the base table, by granting different privileges to the view than the base table.

15. D. The primary use for a tablespace is to organize tables and indexes into manageable units. Some of the manageable operations that you can do to a tablespace include making it read-only or moving it from one database to another.

16. C. An extent is composed of two or more data blocks; a segment is composed of one or more extents, and a datafile houses all these.

17. B. A tablespace can have either dictionary extent management or local extent management.

18. B. Although `binary_float` and `binary_double` are valid datatypes, `binary` is not.

19. A. The options for temporary tables are either `ON COMMIT DELETE ROWS`, with causes the table to flush at the end of each transaction, or `ON COMMIT PRESERVE ROWS`, which causes the table to flush at the end of each session.

20. D. You assign or change comments on a column with the `COMMENT ON COLUMN` statement. The `COMMENT ON TABLE` statement is used to add or change the comment assigned to a table.

Chapter 4

Oracle Net Services

ORACLE DATABASE 10*G*: ADMINISTRATION I EXAM OBJECTIVES COVERED IN THIS CHAPTER:

- ✓ **Controlling the Database**
  - Start and stop the Oracle Listener.
- ✓ **Oracle Net Services**
  - Use Database Control to Create additional listeners.
  - Use Database Control to Create Oracle Net service aliases.
  - Use Database Control to Configure connect time failover.
  - Use Listener features.
  - Use the Oracle Net Manager to configure client and middle-tier connections.
  - Use TNSPING to test Oracle Net connectivity.
  - Describe Oracle Net Services.
  - Describe Oracle Net names resolution methods.

NOTE

Exam objectives are subject to change at any time without prior notice and at Oracle's sole discretion. Please visit Oracle's Training and Certification website (`http://www.oracle.com/education/certification/`) for the most current exam objectives listing.

Networks have evolved from simple terminal-based systems to complex multitiered systems. Today's networks can comprise many computers on multiple operating systems using a wide variety of protocols and communicating across wide geographic areas. We need look no further than the explosion of the Internet to see how networking has matured and what a profound impact networks have on the way we work and communicate.

Although networks have become increasingly complex, they also have become easier to use and manage. For instance, we all take advantage of the Internet without knowing or caring about the components that make this communication possible because the complexity of this huge network is completely hidden from us.

The experienced Oracle database administrator has seen this maturation process in the Oracle network architecture as well. From the first version of SQL*Net to the latest releases of Oracle Net, Oracle has evolved its network strategy and infrastructure to meet the demands of the rapidly changing landscape of network communications.

This chapter highlights the areas that you need to consider when implementing an Oracle network strategy and the responsibilities that you have when managing an Oracle Database 10*g* (Oracle 10*g*) network. We'll also discuss the most common network configurations. The chapter introduces the features of Oracle Net—the connectivity management software that is the backbone of the Oracle network architecture. We'll explain how to configure the main client and server-side components of Oracle Net, and we'll discuss the tools that you have at your disposal to perform these tasks.

Network Design Considerations

Many factors are involved in making network design decisions. First and foremost, you need to understand the design of the Oracle network architecture itself. Oracle Net is flexible and configurable and has the scalability to accommodate a range of network sizes. You can choose from a variety of network configurations that are designed to meet the needs of both small and large organizations. The sections that follow summarize the areas that you need to consider.

Network Complexity Issues

The complexity of the network plays an important role in many of your network design decisions. To determine network complexity, you need to answer the following questions:

- How many clients will the network need to support?
- What type of work will the clients be doing?

- Where are the clients? In complex networks, clients can be geographically dispersed over a wide area.
- What types of clients are going to be supported? Will they be PC-based or terminal-based? Will these be thin clients that do little processing or fat clients that do most of the application processing?
- What is the projected growth of the network?
- Where will the processing take place? Will any middle-tier servers be involved, such as an application server or a transaction server?
- What types of network protocols will be used to communicate between the clients and servers?
- Will Oracle servers have to communicate with other Oracle servers in the enterprise?
- Will the network involve multiple operating systems?
- Do the applications that will be used require any special networking? This is especially important to consider when you are dealing with third-party applications.

Answering these questions will provide you with guidance in determining the size and amount of the network hardware and software required to monitor and manage the network. You will also start to gain insights into the staffing levels and time requirements needed to support the network.

Network Security Issues

Network security has become even more critical as companies expose their systems to larger and larger numbers of users through Internet and intranet connections. To determine the security of a network, you need to answer the following questions:

- Does the organization have any special requirements for secure network connections? What kinds of information will be sent across the Oracle network?
- Can you ensure secure connections across a network without risk of information tampering? This may involve sending the data in a format that makes it tamper-proof and also ensures that the data cannot be captured and read by parties other than the client and the intended Oracle server.
- Is there a need to centralize the authorizations that an individual has to each of the Oracle servers? In large organizations that have many Oracle services, this can be a management and administration issue.

Once you gain an understanding of what the network security requirements are, you should know the following:

- The network hardware and software requirements
- The modifications to applications to meet these security requirements
- The policy and procedure changes within the organization to meet the requirements
- The staffing levels to monitor and manage security

Interfacing Existing Systems with New Systems

Many organizations are faced with the challenge of interfacing legacy systems to new systems. Often these legacy systems contain mission-critical information, and must be able to communicate to new applications as the business evolves. If existing computer systems must communicate with Oracle server networks, you need to answer the following questions:

- Does the application that needs to perform the communication require a seamless, real-time interface?
- Does the existing system use a non-Oracle database such as IBM DB2 or Microsoft SQL Server?
- Will information be transferred periodically from the existing system to the Oracle server? If so, what is the frequency and which transport mechanisms should be used? Will the Oracle server need to send information back to the existing system?
- Do applications need to gather data from multiple sources, including Oracle and non-Oracle databases, simultaneously?
- What are the applications involved that require this interface?
- Will these network requirements necessitate design changes to existing systems?

Answering these questions will give you a better understanding of what systems need to be able to communicate with Oracle Net and to what degree these systems need to be integrated.

Network Responsibilities for the DBA

As a database administrator, you have many design issues to consider when implementing a network of Oracle servers in the enterprise. Here are some of your key responsibilities:

- Understand the network configuration options available and know which options should be used based on the requirements of the organization.
- Understand the underlying network architecture of the organization in order to make informed design decisions.
- Work closely with the network engineers to ensure consistent and reliable connections to the Oracle servers.
- Understand the tools available for configuring and managing the network.
- Troubleshoot connection problems on the client, middle tier, and server.
- Ensure secure connections and use the available network configurations, when necessary, to attain higher degrees of security for sensitive data transmissions.
- Stay abreast of trends in the industry and changes to the Oracle architecture that might have an impact on network design decisions.

Network Configurations

You can select from three basic types of network configurations when designing an Oracle infrastructure:

- Single tier
- Two tier
- *n*-tier

Single tier is the simplest type. It has been around for years and is characterized by the use of terminals for serial connections to the Oracle server. The two-tier configuration is also referred to as the client/server architecture, and more recently the *n*-tier architecture has been introduced. Let's take a look at each of these configuration alternatives.

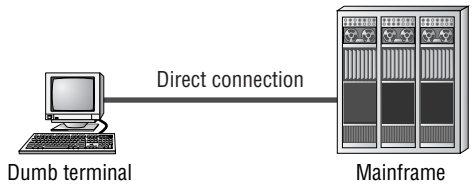
Single-Tier Architecture

Single-tier architecture was the standard for many years before the birth of the PC. Applications using single-tier architecture are sometimes referred to as green-screen applications because most of the terminals that used them, such as the IBM 3270, had green screens. Single-tier architecture is commonly associated with mainframe-type applications.

This architecture is still in use today for many mission-critical applications, such as order processing and fulfillment and inventory control, because it is the simplest architecture to configure and administer. Because the terminals are directly connected to the host computer, the complexities of network protocols and multiple operating systems don't exist.

When a single-tier architecture is used, users interact with the database using terminals, which are nongraphical, character-based devices. Figure 4.1 shows an example of the single-tier architecture. In this type of architecture, client terminals are directly connected to larger server systems such as mainframes. All the intelligence exists on the mainframe, and all processing takes place there. Simple serial connections also exist on the mainframe. Although no complex network architecture is necessary, a single-tier architecture is somewhat limiting in terms of scalability and flexibility. Because all the processing must take place on the server, the server can become the bottleneck to increasing performance.

FIGURE 4.1 Single-tier architecture



Two-Tier Architecture

Two-tier architecture gained popularity with the introduction of the PC and is commonly referred to as client/server computing. In a two-tier environment, clients connect to servers over a network using a network protocol, which is the agreed-upon method for the client to communicate with the server. TCP/IP (Transmission Control Protocol/Internet Protocol) is a popular network protocol and has become the de facto standard of network computing. Whether you choose TCP/IP or some other network protocol, both the client and the server must be able to understand it. Figure 4.2 shows an example of a two-tier architecture.

This architecture has definite benefits over single-tier architecture. First, client/server computing introduces the graphical user interface (GUI). This interface is easier to understand and learn, and it offers more flexibility than the traditional character-based interfaces of the single-tier architecture. Also, two-tier architecture allows the client computer to share the application process load. To a certain degree, this reduces the processing requirements of the server.

The two-tier architecture does have some faults, even though at one time, this configuration was thought to be the panacea of all networking architectures. Unfortunately, the main problem—that being scalability—persists. Notice that the term *client/server* contains a slash (/). The slash represents the invisible component of the two-tier architecture and the one that is often overlooked: the network! The limitation of client/server computing is one of scalability.

When prototyping projects, many developers fail to consider the network component and soon find out that what worked well in a small environment does not scale effectively to larger, more complex systems. The two-tier architecture model was subject to a great deal of redundancy because application software was required on every desktop. As a result, many companies end up with bloated PCs and large servers that still do not perform adequately. What is needed is a more scalable model for network communications. That is what *n*-tier architecture provides.

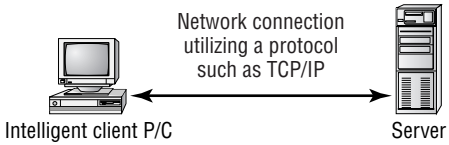
N-Tier Architecture

N-tier architecture is the next logical step after two-tier architecture. Instead of dividing application processing work between a client and a server, you divide the work among three or more machines. The n-tier architecture introduces *middleware* components, such as application servers or web servers, situated between the client and the database server, which can be used for a variety of tasks, including the following:

- Moving data between machines that work with different network protocols
- Serving as firewalls that can control client access to the servers
- Offloading processing of the business logic from the clients and servers to the middle tier
- Executing transactions and monitoring activity between clients and servers to balance the load among multiple servers
- Acting as a gateway to bridge existing systems to new systems

The Internet is an example of the ultimate *n*-tier architecture, with the user's browser providing a consistent presentation interface. This common interface means less training of staff and also increases the potential reuse of client-side application components.

FIGURE 4.2 Two-tier architecture



N-tier architecture is rapidly becoming the architecture of choice for enterprise networks. This model is scalable and divides the tasks of presentation, business logic and routing, and database processing among many machines, which means that this model accommodates large applications. Many factors are driving *n*-tier computing, such as the Internet and Oracle grid computing, which uses a large number of back-end processors to scale database services and connectivity.

By reducing the processing load on the database servers, those servers can do more work with the same number of resources. Also, the transaction servers can balance the flow of network transactions intelligently, and application servers can reduce the processing and memory requirements of the client (see Figure 4.3).

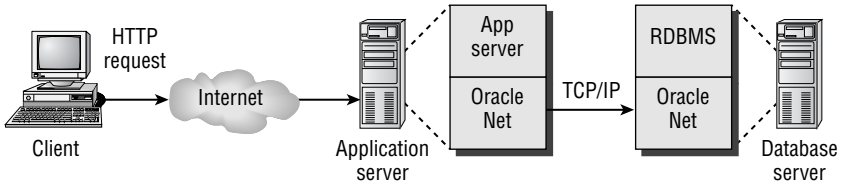
An Overview of Oracle Net Features

Oracle Net is the glue that bonds the Oracle network together. It is responsible for handling client-to-server and server-to-server communications, and it can be configured on the client, the middle-tier application, web servers, and the Oracle server.

Oracle Net manages the flow of information in the Oracle network infrastructure. First, it establishes the initial connection to the Oracle server, and then it acts as the messenger, passing requests from the client back to the server or passing them between two Oracle servers. Oracle Net handles all negotiations between the client and server during the client connection.

In addition to functioning as an information manager, Oracle Net supports the use of middleware products such as Oracle Application Server and Oracle Connection Manager. These products allow *n*-tier architectures to be used in the enterprise, which increases the flexibility and performance of application designs.

FIGURE 4.3 Connection requests in *n*-tier architecture



To provide a further understanding of the features of Oracle Net, the following sections discuss in detail the five categories of networking solutions that Oracle Net addresses:

- Connectivity
- Manageability
- Scalability
- Security
- Accessibility

Connectivity

There are many ways in which a client can interact with an Oracle database. A client may be running a PC-based application or a dumb terminal application, or perhaps the client is connecting to the database via the Internet. Let's take a look at how Oracle supports connectivity to the database through these and other interfaces.

Multi-Protocol Support

Oracle Net supports a wide range of industry-standard protocols such as TCP/IP and named pipes. This support is handled transparently and allows Oracle Net to connect to a wide range of computers and a wide range of operating environments.

Multiple Operating Systems

Oracle Net can run on many operating system platforms, from Windows XP, to all variants of Unix, to large mainframe-based operating systems. This range allows users to bridge existing systems to other Unix or PC-based systems, which increases the data access flexibility of the organization without making wholesale changes to the existing systems.

Java and JDBC

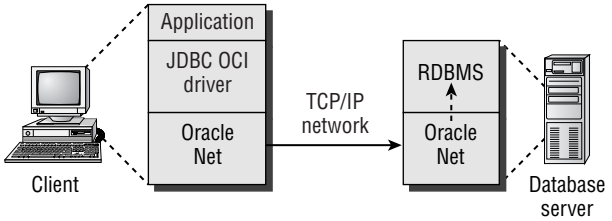
Applications written in Java can take advantage of the Java Database Connectivity (JDBC) drivers provided with Oracle to connect to an Oracle server. The two basic types of JDBC drivers are JDBC Oracle Call Interface (OCI) and JDBC thin.

The JDBC Oracle Call Interface driver is a client-side installed driver that is used if the Java application is resident on a client computer. This driver is also called a type II driver because the driver software is installed on the computer that is using the application. It uses OCI to interact with the Oracle Net infrastructure. Figure 4.4 shows how a client and server communicate when using a JDBC OCI connection.

In this example, the Java application installed on the client uses the JDBC OCI driver and Oracle database server. When an application makes a database request, it uses the JDBC OCI driver to translate the JDBC calls and send them to Oracle Net. Oracle Net is used on both the client and server to broker all communications between the two endpoints.

The JDBC thin driver is written entirely in Java and, as such, is platform independent. It does not have to be installed on a client computer (hence, the term *thin driver*). The driver interfaces directly with a layer of the Oracle Net infrastructure called the Two-Task Common Layer.

FIGURE 4.4 Oracle JDBC OCI connection



Manageability

Oracle Net provides a variety of features that allow you to manage the components of an Oracle network. Let's review the key manageability features of Oracle Net.

Web Applications

Oracle Net supports a variety of connectivity solutions from a web browser interface. Connections can be made through a middle-tier web or application server or directly from a web browser to an Oracle service.

When a middle-tier solution is used, the web browser uses HTTP to contact a database service and request information. Typically, an application or web server receives this request and hands it off to Oracle Net, which manages the connection between the web server and the database server. Once the database server receives the connection request, the request is processed and passed back to the web server. The web server then sends the response back to the client's web browser. This type of request fulfillment requires that the middle-tier application server be loaded with the Oracle Net software, but the client does not require any additional software.

Oracle also supports web connectivity directly from a web client. For example, a Java applet running within a web browser can use a JDBC driver to connect directly to an Oracle server without the need for an application or web server.

Location Transparency

Oracle Net provides the infrastructure to manage the database location. This is important especially in large organizations that support many databases and clients. Each database in the organization is represented as one or more services. Database services are defined by one or more service names. The actual definition of the service names is managed within Oracle Net. The definition holds information about the type and location of the service on the network. This layer of abstraction provides location transparency to the client and centralizes the management of connection information within Oracle Net, which simplifies the job of managing the network.

Directory Naming

Directory Naming allows service names to be resolved through a centralized naming repository. The central repository takes the form of a Lightweight Directory Access Protocol (LDAP)–compliant server. LDAP is a protocol and language that defines a standard method for storing, identifying, and retrieving services. It provides a simplified way to manage directories of information, whether this information is about users in an organization or Oracle services connected to a network. The LDAP server allows for a standard form of managing and resolving names in an Oracle environment. The quality of these services excels because LDAP provides a single, industry-standard interface to a directory service such as Oracle Internet Directory (OID). By using OID, you ensure security and reliability of the directory information because information is stored in the Oracle database.

As of Oracle 10*g*, Directory Naming is the preferred method of centralized naming within an Oracle environment, replacing the Oracle Names Server, which is no longer supported for centralized naming.

Scalability

Many enterprise systems are growing rapidly, supporting larger and larger databases and user communities. Your network capabilities need to be able support this growth. Oracle Net provides features that allow you to expand your network reach and maximize your system resources to meet these demands.

Oracle Shared Server

Oracle Shared Server is an optional configuration of the Oracle server that allows support for a large number of concurrent connections without increasing physical resource requirements. This is accomplished by sharing resources among groups of users.

Directory Services: Oracle Internet Directory

The OID is an LDAP 3–compliant directory service that provides the repository and infrastructure needed to enable a centralized naming solution using Directory Naming. OID is compatible with older releases of Oracle such as Oracle8*i* and Oracle9*i*. In Oracle 10*g*, the OID runs as an application. The OID service can run on a remote server, and it can communicate with the Oracle server using Oracle Net. The OID is a scalable architecture, and it provides mechanisms for replicating service information among other Oracle servers.

OID also provides security in a number of ways. First, it can be integrated into a Secure Sockets Layer (SSL) environment to ensure user authentication. Also, an administrator can maintain policies that grant or deny access to services. These policies are defined for entities within the OID tree structure.

Oracle Shared Server is discussed in detail in Chapter 5, "Oracle Shared Server."

Connection Manager

Oracle Connection Manager is a middleware solution that provides three additional scalability features:

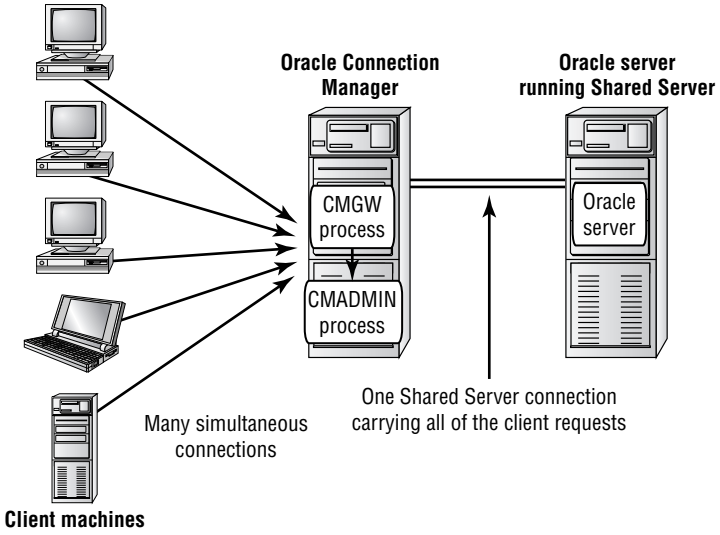
Multiplexing Connection Manager can group many client connections and send them as a single multiplexed network connection to the Oracle server. This reduces the total number of network connections that the server has to manage.

Network access You can configure Connection Manager with rules that restrict access by IP address. You can set up this rules-based configuration to accept or reject client connection requests. Also, connections can be restricted by point of origin, destination server, or Oracle server.

Cross-protocol connectivity This feature allows clients and servers that use different network protocols to communicate. Connection Manager acts as a translator, providing two-way protocol conversion.

Oracle Connection Manager is controlled by a set of background processes that manage the communications between clients and servers. Figure 4.5 provides an overview of the Connection Manager architecture.

FIGURE 4.5 Connection Manager architecture



Security

The threat of data tampering and database security is an issue of major concern in many organizations, as network systems continue to grow in number and complexity and as users gain increasing access to systems. Sensitive business transactions are being conducted with greater frequency and, in many cases, are not protected from unauthorized tampering or message interception. Oracle Net is capable of providing organizations with a secure network environment to conduct business transactions.

Advanced Security

Oracle Advanced Security, formerly known as the Advanced Security Option and the Advanced Networking Option, not only provides the tools necessary to ensure secure transmissions of sensitive information, but it also provides mechanisms to confidently identify and authenticate users in the Oracle enterprise.

When configured on the client and the Oracle server, Oracle Advanced Security supports secured data transactions by encrypting and optionally checksumming the transmission of information that is sent in a transaction. Oracle supports encryption and checksumming by taking advantage of industry-standard algorithms, such as RSA RC4, Standard DES and Triple DES, and MD5 checksumming. These security features ensure that data transmitted from the client has not been altered during transmission to the Oracle server.

Oracle Advanced Security also gives you the ability to authenticate users connecting to the Oracle servers. In fact, a number of authentication features ensure that users really are who they claim to be. These are offered in the form of token cards, which use a physical card and a user-identifying PIN number to gain access to the system; the biometrics option, which uses fingerprint technology to authenticate user connection requests; public key; and certificate-based authentication.

Another feature of Oracle Advanced Security is the ability to have a single sign-on mechanism for clients. Single sign-on is accomplished with a centralized security server that allows the user to connect to any of the Oracle services in the enterprise using a single user ID and password. Oracle leverages the industry-standard features of Kerberos to enable these capabilities. This greatly simplifies the privilege matrix that administrators must manage when they are dealing with large numbers of users and systems.

Kerberos is an authentication mechanism based on the sharing of secrets between two systems.

Firewall Support

Firewalls are an important security mechanism in corporate networks. Firewalls are generally a combination of hardware and software used to control network traffic and prevent intruders from compromising corporate network security. Firewalls fall into two broad categories:

IP-Filtering Firewalls *IP-filtering firewalls* monitor the network packet traffic on IP networks and filter out packets that either originated or did not originate from specific groups of machines.

The information contained in the IP packet header is interrogated to obtain this information. Vendors of this type of firewall include Network Associates and Axent Communications.

Proxy-Based Firewalls *Proxy-based firewalls* prevent information from outside the firewall from flowing directly into the corporate network. The firewall acts as a gatekeeper, inspecting packets and sending only the appropriate information through to the corporate network. This prevents any direct communication between clients outside the firewall and applications inside the firewall. Check Point Software Technologies and Cisco are examples of vendors that market proxy-based firewalls.

Oracle works closely with the vendors of both types of firewalls to ensure support of database traffic through these types of mechanism. Oracle supplies the Oracle Net Application Proxy Kit to the firewall vendors. This product can be incorporated into the firewall architecture to allow database packets to pass through the firewall and still maintain a high degree of security.

Accessibility

In many organizations, workers need to be able to communicate across a variety of systems and databases. They spend a lot of time bringing together data from different systems. The accessibility features of Oracle Net have capabilities that allow you to communicate with non-database data sources. This ability opens up new opportunities to provide customers with accurate and timely information.

Real World Scenario

Know Thy Firewall

It is important to understand your network infrastructure, the network routes that you are using to obtain database connections, and the type of firewall products that you are using. In more than one situation, I've seen firewalls cause connectivity issues between a client and an Oracle server.

For instance, a small patch was applied to a firewall when I was working as a DBA for one of my former employers. In this case, employees started experiencing intermittent disconnects from the Oracle database. After many days of investigation and network tracing, we pinned down the exact problem. We then contacted the firewall vendor, who sent us a new patch to apply that corrected the problem.

In another instance, I was working as a DBA for a large corporate client. The development staff started experiencing a similar connection problem. It turned out that the networking routes for the development staff had been modified to connect through a new firewall, with connections timing out after 20 minutes. This timeout was too short for this department. Increasing the timeout parameter solved the problem.

These are examples of the types of network changes that you need to be aware of to avoid unnecessary downtime and to avoid wasting staff time and resources.

Heterogeneous Services

Heterogeneous Services provide the ability to communicate with non-Oracle databases and services. These services allow organizations to leverage and interact with their existing data stores without having to necessarily move the data to an Oracle server.

The suite of Heterogeneous Services comprises the *Oracle Transparent Gateway* and *Generic Connectivity*. These products allow Oracle to communicate with non-Oracle data sources in a seamless configuration. Heterogeneous Services also integrate existing systems with the Oracle environment, which allows you to leverage your investment in those systems. These services also allow for two-way communication and replication from Oracle data sources to non-Oracle data sources.

Transparent Gateway seamlessly extends the reach of Oracle to non-Oracle data stores, which allows you to treat non-Oracle data sources as if they were part of the Oracle environment. In fact, the user is not even aware that the data being accessed is coming from a non-Oracle source. This can significantly reduce the time and investment necessary to transition from existing systems to the Oracle environment. Transparent Gateway fully supports SQL and the Oracle transaction control features, and it currently supports access to more than 30 non-Oracle data sources.

Generic Connectivity provides a set of agents, which contain basic connectivity capabilities. It also provides a foundation so that you can custom-build connectivity solutions using standard OLE Database, Microsoft's interface to data access. OLE DB requires an ODBC driver to interface to the agents. You can also use ODBC as a stand-alone connection solution. For example, with the proper Oracle ODBC driver, you can access an Oracle database from programs such as Microsoft Excel. (You can obtain these drivers from Oracle or third-party vendors.) Because these drivers are generic in nature, they do not provide as robust an interface to external services as does the Transparent Gateway.

External Procedures

In some development efforts, interfacing with procedures that reside outside the database may be necessary. These procedures are typically written in a third-generation language, such as C. Oracle Net provides the ability to invoke such external procedures from Oracle PL/SQL callouts. When a call is made, a process is started that acts as an interface between Oracle and the external procedure. This callout process defaults to the name *extproc*. The listener is then responsible for supplying information, such as a library or procedure name and any parameters, to the called procedure. These programs are then loaded and executed under the control of the extproc process.

Configuring Oracle Net on the Server

Now that you understand the basic features that Oracle Net provides, you need to understand how to configure the major components of Oracle Net. You must configure Oracle Net on the server in order for client connections to be established. This section will focus on how to configure the network elements of the Oracle server. It will also describe the types of connection methods that Oracle Net supports. We will then discuss how to manage Oracle Net on the server and troubleshoot connections from the server if clients experience connection problems.

Understanding the Oracle Listener

The Oracle *listener* is the main server-side Oracle networking component that allows connections to be established between client computers and an Oracle database. You can think of the listener as a big ear that listens for connection requests to Oracle services.

The type of Oracle service being requested is part of the connection descriptor information supplied by the process requesting a connection, and the service name resolves to an Oracle database. The listener can listen for any number of databases configured on the server, and it is able to listen for requests being transported on a variety of protocols. A client connection can be initiated from the same machine that the listener resides on, or it may come from some remote location.

The listener is controlled by a centralized file called *listener.ora*. Though only one `listener.ora` file is configured per machine, there may be numerous listeners on a server, and this file contains all the configuration information for every listener configured on the server. If multiple listeners are configured on a single server, they are usually set up for failover purposes or to balance connection requests and minimize the burden of connections on a single listener.

Multiple listeners may also be used when you configure Oracle Real Application Clusters. In a Real Application Clusters environment, these listeners typically span multiple server nodes, and all the corresponding nodes connect to a single database. Later in this chapter, in the section "Additional Configurations When Using Multiple Listeners," we'll discuss other reasons to configure multiple listeners.

The content and structure of the `listener.ora` file is discussed later in this chapter, in the section "Managing Oracle Listeners."

Every listener is a named process that runs on either a middle-tier server or the database server. The default name of the Oracle listener is `LISTENER`, and it is typically created when you install Oracle. If you configure multiple listeners, each has a unique name. The following is an example of the contents of the `listener.ora` file:

```
# listener.ora Network Configuration File:
     D:\oracle\ora10g\NETWORK\ADMIN\listener.ora
# Generated by Oracle configuration tools.

SID_LIST_LISTENER =
  (SID_LIST =
    (SID_DESC =
      (SID_NAME = PLSExtProc)
      (ORACLE_HOME = D:\oracle\ora10g)
      (PROGRAM = extproc)
    )
    (SID_DESC =
      (GLOBAL_DBNAME = ORCL.COM)
```

```
      (ORACLE_HOME = d:\oracle\ora10g)
      (SID_NAME = ORCL)
    )
  )

LISTENER =
  (DESCRIPTION_LIST =
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = TCP)(HOST = mjworn.corp.testenv.net)(PORT = 1521))
    )
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC))
    )
  )
```

Now that you have a basic understanding of the Oracle Listener, let's explore the main function of the Listener, responding to client connection requests.

How Do Listeners Respond to Connection Requests?

A listener can respond to a client request for a connection in several ways. The response depends on several factors, such as how the server-side network components are configured and what type of connection the client is requesting. The listener then responds to the connection request in one of two ways.

The listener can spawn a new process and pass control of the client session to the process. In a *dedicated server* environment, every client connection is serviced by its own server-side process. Server-side processes are not shared among clients. Two types of dedicated connection methods are possible: direct and redirect. Each method results in a separate process that handles client processing, but the mechanics of the actual connection initiation process are different. For remote clients to use dedicated connections, the listener process must be running on the same physical server as the database or databases for which it is listening.

The listener can also pass control of a connection request to a dispatcher. This type of connection takes place in an Oracle Shared Server environment. There are also two types of connection methods when using Oracle Shared Server: direct and redirect.

Let's take a look at each of these connection method types.

NOTE

We'll discuss Oracle Shared Server in detail in Chapter 5.

Dedicated Connections: Direct Handoff Method

Direct handoff connections are possible when the client and database exist on the same server. For example, a direct handoff method is used when the client connection request originates from the same machine on which the listener and database are running.

Another name for direct handoff connections is bequeath connections.

The following steps, which show the connection process for the bequeath connections, are illustrated in Figure 4.6:

1. The client contacts the Oracle listener after resolving the service name.
2. The listener starts a dedicated process, and the client connection inherits the dedicated server process network connect endpoint from the listener.
3. The client now has an established connection to the dedicated server process.

Dedicated Connections: Redirect Method

Redirect connections occur in a dedicated server environment when the client exists on a machine that is separate from the listener and database server. The listener must inform the client of the address of the spawned process in order for the process to contact the newly created dedicated server process.

The following steps, which show the connection process for redirect connections in a dedicated server environment, are illustrated in Figure 4.7:

1. The client contacts the Oracle listener after resolving the service name.
2. The listener starts a dedicated process.
3. The listener sends an acknowledgment back to the client with the address of the dedicated server connect endpoint on the database server to which the client will connect.
4. The client establishes a connection to the dedicated server connect endpoint.

FIGURE 4.6 Dedicated connection: direct handoff method

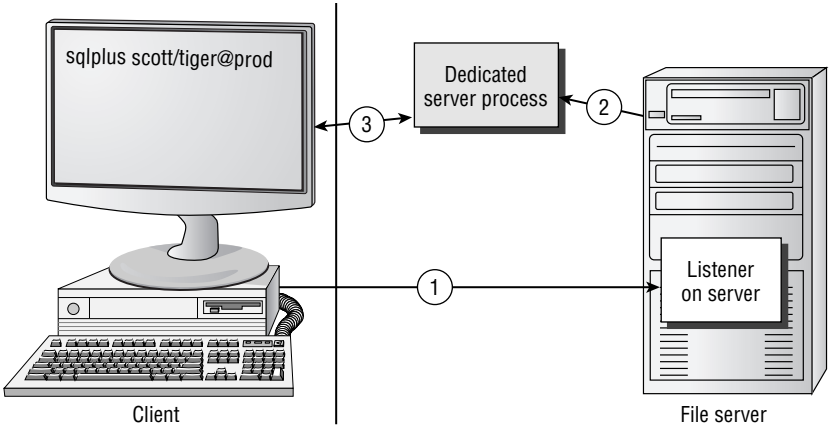
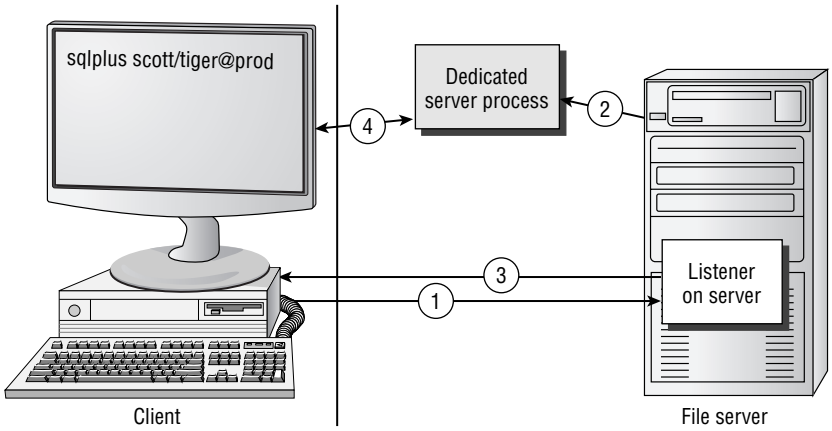


FIGURE 4.7 Dedicated connections: redirect method

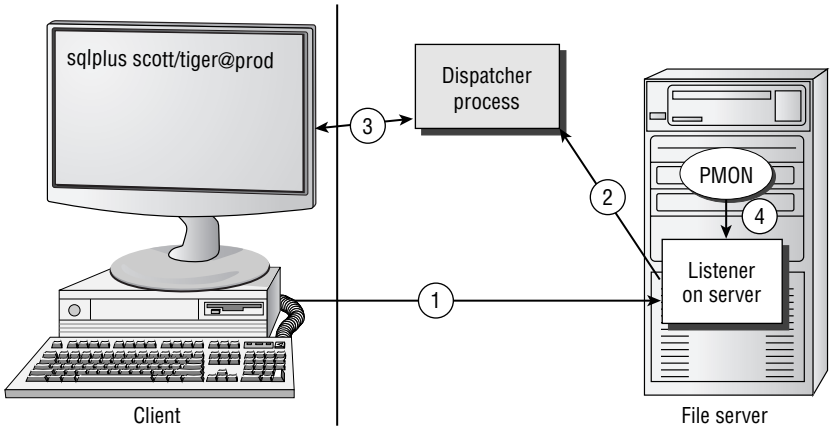


Oracle Shared Server: Direct Handoff Method

When you are using Oracle Shared Server, the client connection can also be established using a direct handoff method. This would be the case, for example, when the client request originates from the same machine as the listener and the database are running. Figure 4.8 outlines the connection steps when using Oracle Shared Server and the direct handoff method:

1. The client contacts the Oracle listener after resolving the service name.
2. The Oracle Listener passes the connection request to the least loaded dispatcher.
3. The client now has an established connection to the dispatcher process.

FIGURE 4.8 Oracle Shared Server: direct handoff method



Oracle Shared Server: Redirect Method

The listener can also redirect the user to a server process or a dispatcher process when using Oracle Shared Server. This type of connection can occur when the operating system does not directly support direct handoff connections or the listener is not on the same physical machine as the Oracle server.

The following steps are illustrated in Figure 4.9:

1. The client contacts the Oracle server after resolving the service name.
2. The listener sends information back to the client, redirecting the client to the dispatcher port. The original network connection between the listener and the client is disconnected.
3. The client then sends a connect signal to the server or dispatcher process to establish a network connection.
4. The dispatcher or server process sends an acknowledgment back to the client.
5. PMON (process monitor) sends information to the listener about the number of connections being serviced by the dispatchers. The listener uses this information to maintain consistent loads between the dispatchers.

Managing Oracle Listeners

You can configure the server-side listener files in a number of ways. As part of the initial Oracle installation process, the installer prompts you to create a default listener. If you choose this method, the installer uses the set of screens that are a part of the Oracle Net Configuration Assistant to do the initial listener configuration. Figure 4.10 shows an example of the opening screen for this assistant.

FIGURE 4.9 Oracle Shared Server: redirect connection method

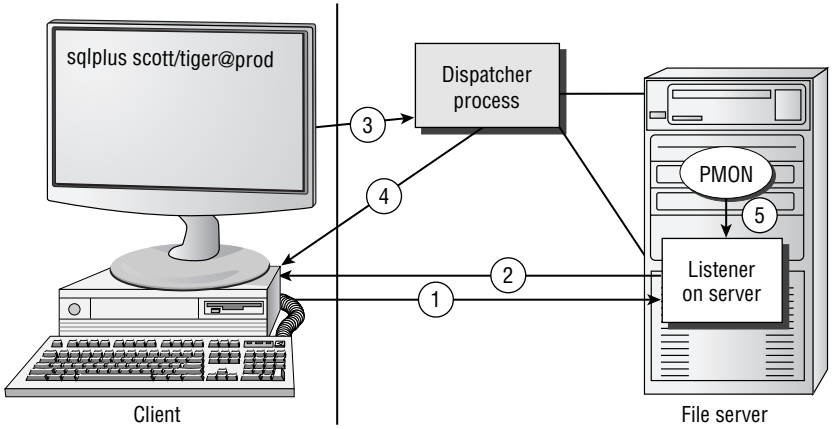


FIGURE 4.10 Oracle Net Configuration Assistant opening screen

If you want to set up more than just basic configurations of Oracle network files, you will have to use the Oracle Net Manager, the web-based tool Oracle Enterprise Manager (EM), or the command-line facility `lsnrctl`. In the next few sections, you will learn how to use these tools to configure the server-side network files.

Managing Listeners with Oracle Net Manager

Oracle Net Manager is a tool you can use to create and manage most client- and server-side configuration files. The Oracle Net Manager has evolved from the Oracle7 tool, Network Manager, to the latest Oracle 10*g* version. Throughout this evolution, Oracle has continued to enhance the functionality and usability of the tool.

If you are using a Windows environment, you can start the Oracle Net Manager by choosing Start ➢ Programs ➢ *Your Oracle 10g Programs choice* ➢ Configuration and Migration Tools ➢ Oracle Net Manager. In a Unix environment, you can start it by running `./netmgr` from your `$ORACLE_HOME/bin` directory.

Figure 4.11 shows an example of the Oracle Net Manager opening screen.

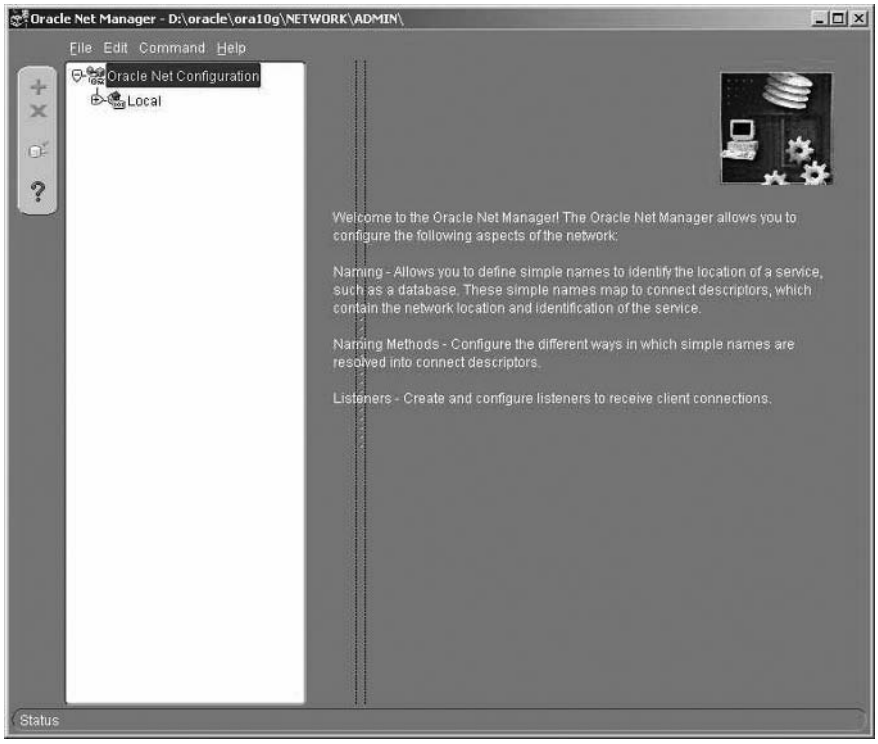
Configuring Listener Services Using the Oracle Net Manager

Oracle Net Manager provides an easy-to-use graphical interface for configuring most of the network files that you will be using. By using Oracle Net Manager, you can ensure that the files are created in a consistent format, which will reduce the potential for connection problems.

When you first start the Oracle Net Manager, the opening screen displays a tree structure with a top level called Oracle Net Configuration. If you click the plus (+) sign next to this icon, you will see the Local folder. The choices under the Local folder relate to different network configuration files. Here are the network file choices and what each configures:

Icon	File Configured
Profile	`sqlnet.ora`
Service Naming	`Tnsnames.ora`
Listeners	`Listener.ora`

FIGURE 4.11 The opening screen for Oracle Net Manager



Creating the Listener

Earlier, we said that, by default, Oracle creates a listener called `LISTENER` when it is initially installed. The default settings that Oracle uses for the `listener.ora` file are shown here:

Section of the File	Setting
Listener Name	`LISTENER`
Port	`1521`
Protocols	`TCP/IP and IPC`
Host Name	`Default Host Name`
SID Name	`Default Instance`

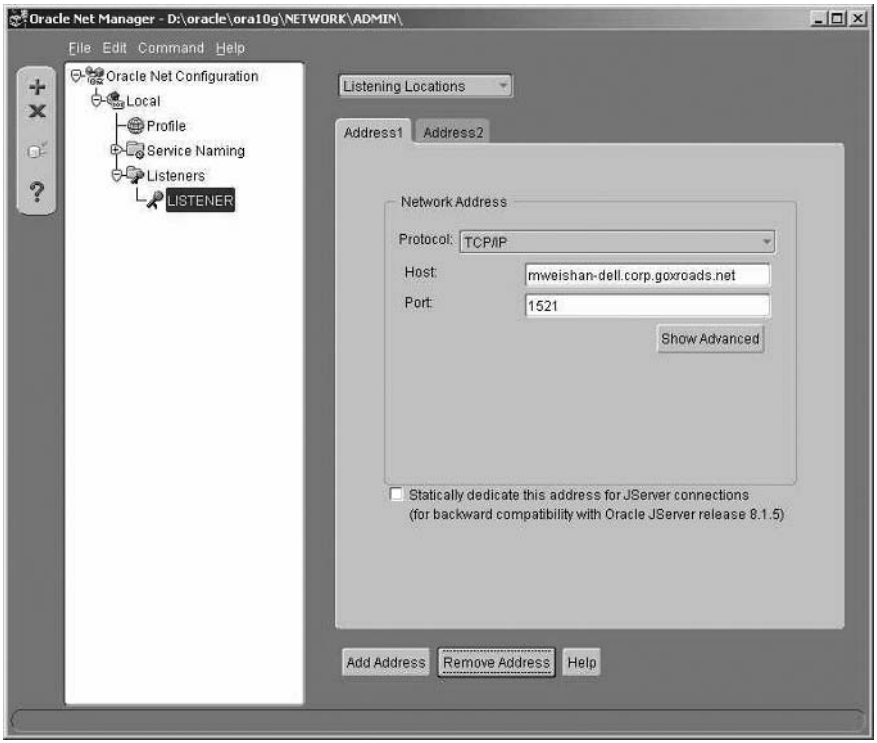
You can use Oracle Net Manager to create a non-default listener or change the definition of existing listeners. The Oracle Net Manager has a wizard interface for creating most of the basic network elements, such as the `listener.ora` and `tnsnames.ora` files.

Follow these steps to create the listener:

1. Click the plus (+) sign next to the Local icon.
2. Click the Listeners folder.
3. Click the plus sign icon or choose Edit ➢ Create to open the Choose Listener Name dialog box.

4. The Oracle Net Manager defaults to `LISTENER` or to `LISTENER1` if the default listener is already created. Click OK if this is correct, or enter a new name and then click OK to open the Listening Locations screen, as shown in Figure 4.12.
5. To configure the listening locations, click the Listening Locations drop-down list and make your selection. Then click the Add Address button at the bottom of the screen to open a new window.

FIGURE 4.12 The Listening Locations screen



The prompts on this screen depend on your protocol. By default, TCP/IP information is displayed. If you are using TCP/IP, the Host and Port fields are filled in for you. The *host* is the name of the machine in which the listener is running, and the *port* is the listening location for TCP/IP connections. The default value for the port is 1521.

6. To save your information, choose File ➢ Save Network Configuration, and then look in the directory where the files was saved.

You can add additional listeners by following these steps. Listeners must have unique names and listen on separate ports, so assign the listener a new name and a new port (1522, for example). You also must assign service names to the listener. We'll see how to add service information in the next section.

> **TIP** You always know where the files are stored by looking at the top banner of the Oracle Net Manager screen.

The Oracle Net Manager actually creates three files in this process: `listener.ora`, `tnsnames.ora`, and `sqlnet.ora`. The `tnsnames.ora` file does not contain any information. The `sqlnet.ora` file may contain a few entries at this point, but you can ignore them for the time being. The `listener.ora` file contains information as shown in the following code.

```
# listener.ora Network Configuration File:
     D:\oracle\ora10g\network\admin\listener.ora
# Generated by Oracle configuration tools.

LISTENER =
  (DESCRIPTION_LIST =
    (DESCRIPTION =
      (ADDRESS_LIST =
        (ADDRESS = (PROTOCOL = TCP)(HOST = mjworn.corp.testenv.net)
     (PORT = 1521))
      )
      (ADDRESS_LIST =
        (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC))
      )
    )
  )
```

> **NOTE** We'll discuss the structure and content of the `listener.ora` file later in this chapter.

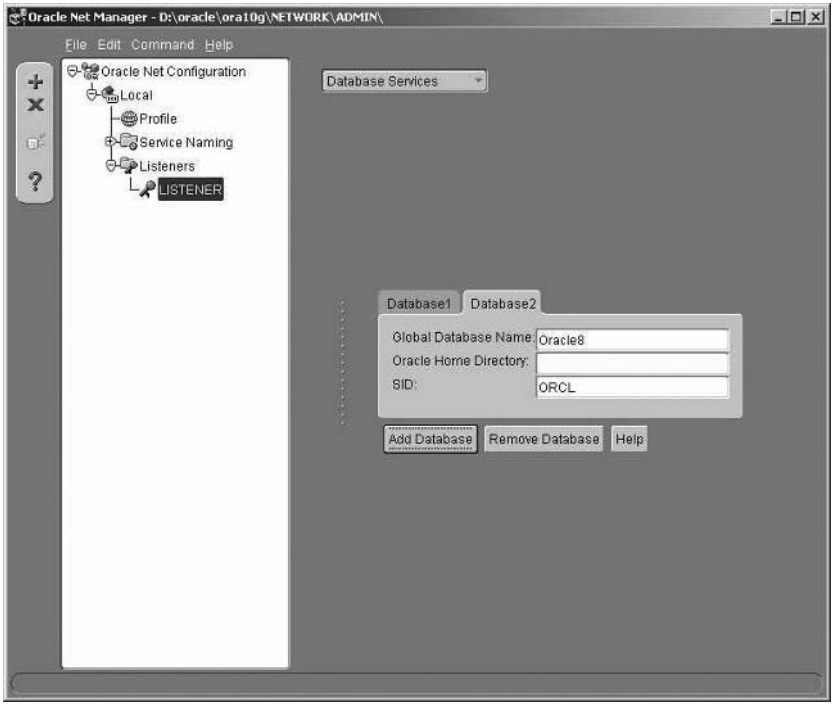
Adding Service Name Information to the Listener

After you create the listener with the name, protocol, and listening location information, you can define the network services to which the listener is responsible for connecting. This is called *static service registration*, because Oracle is not automatically registering the service with the listener. In releases of Oracle prior to Oracle 8*i*, static service registration was the only method to associate services with a listener.

A listener can listen to an unlimited number of network service names. Follow these steps to add the service name information:

1. To select the listener to configure, click the Listeners icon and highlight the name of the listener that you want to configure.
2. From the drop-down list at the top right of the screen, select Database Services.
3. Click the Add Database button at the bottom of the screen. This opens the window that allows you to add the database (see Figure 4.13).
4. Enter values in the Global Database Name, Oracle Home Directory, and SID fields. The entries for SID and Global Database Name are the same if you are using a flat naming convention.
5. Choose File ➢ Save to save your configuration.

FIGURE 4.13 The Database Services screen



Here is an example of the completed `listener.ora` file:

```
# listener.ora Network Configuration File:
     D:\oracle\ora10g\NETWORK\ADMIN\listener.ora
# Generated by Oracle configuration tools.

SID_LIST_LISTENER =
  (SID_LIST =
    (SID_DESC =
      (SID_NAME = PLSExtProc)
      (ORACLE_HOME = D:\oracle\ora10g)
      (PROGRAM = extproc)
    )
    (SID_DESC =
      (GLOBAL_DBNAME = ORCL.COM)
      (ORACLE_HOME = d:\oracle\ora10g)
      (SID_NAME = ORCL)
    )
  )

LISTENER =
  (DESCRIPTION_LIST =
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = TCP)(HOST = mjworn.corp.testenv.net)(PORT = 1521))
    )
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC))
    )
  )
```

Table 4.1 lists and describes each of the `listener.ora` parameters for the Listening Location section of the `listener.ora` file.

TABLE 4.1 Parameters for the Listening Location Section of *listener.ora*

Parameter	Description
`LISTENER`	Indicates the starting point of a listener definition. This is actually the name of the listener being defined. The default name is `LISTENER`.
`DESCRIPTION`	Describes each of the listening locations.